

Assignment 02 – Synchronization & Explicit Waits

1. Module Name

Module 1 – Automation with Selenium

2. Experiment Title

Synchronization & Explicit Waits

3. Problem Statement

Navigate to a dynamic page, trigger loading of content, and wait for the required text to become visible without using `time.sleep()`. Use `WebDriverWait` and `expected_conditions`.

4. Objective

To understand synchronization in Selenium and demonstrate reliable explicit waiting for dynamically loaded elements.

5. Tools, Software and Concepts Used

- Python 3.x
- Selenium WebDriver
- Google Chrome
- Visual Studio Code
- WebDriverWait
- expected_conditions
- visibility_of_element_located()
- By.XPATH
- assert statement

6. Source Code / Implementation Steps

1. Open the Dynamic Loading page.
2. Click the Start button.
3. Create a WebDriverWait object with a 10-second timeout.
4. Wait until Hello World! becomes visible.
5. Read and verify the displayed text.
6. Print the execution result and close the browser.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
```

```
driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
driver.maximize_window()

print("Dynamic Loading page opened.")

start_button = driver.find_element(By.XPATH, "//button[text()='Start']")
start_button.click()

print("Start button clicked.")

wait = WebDriverWait(driver, 10)

hello_text = wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//div[@id='finish']/h4")
    )
)

result_text = hello_text.text
print("Displayed text:", result_text)

assert result_text == "Hello World!"

print("Text verification successful.")
print("Assignment 02 completed successfully.")

input("Press ENTER to close the browser...")

finally:
    driver.quit()
```

7. Output / Labelled Screenshots

- Screenshot 1 – Dynamic Loading page
- Screenshot 2 – Hello World! displayed after the wait
- Screenshot 3 – VS Code terminal showing successful execution

Screenshots are stored in the Screenshots folder for this assignment.

8. Result

The dynamic content was handled successfully using `WebDriverWait` and `visibility_of_element_located()`. The expected Hello World! text was detected and verified without using `time.sleep()`.

9. Observation

Explicit waits synchronize the automation with the actual state of the web page. Selenium continues as soon as the expected condition is satisfied, up to the configured timeout.

10. Conclusion

The experiment successfully demonstrated synchronization using explicit waits in Selenium and provided a reliable method for handling dynamically loaded web elements.